

WALLACE Jacob

BAULARD Nathan



P1RV

Sujet 3 : Jeu en OpenGL

Rapport final

Sommaire :

1. Introduction du projet
2. Libraries
3. Monde et affichage
4. UI
5. Shaders
6. Joueur et fonctionnalités
7. Interactions avec le monde
8. IA Ennemi
9. Conclusion

I Introduction

Le 6 octobre 2023, nous avons commencé le projet P1RV dans le cadre de notre formation de l'option Réalité Virtuelle de l'Ecole Centrale de Nantes. Intitulé "*Jeu en OpenGL*", il consistait à développer un jeu vidéo en C++, principalement à l'aide de la librairie OpenGL. Le développement se déroulait sur 3 mois, de début octobre à début janvier, et sert à utiliser et à prolonger ce que nous avons vu en cours sur ces premiers mois. Ce projet s'accompagne de rapports hebdomadaires à nos encadrants, qui feront compte du travail réalisé chaque semaine, des difficultés rencontrées, des modifications dans le planning et des tâches à réaliser pour la semaine suivante

Le jeu vidéo à développer étant libre, nous avons commencé par choisir le type de jeu que nous souhaitons développer. Le choix s'est vite porté sur un *Halo-Like*. Même s'il est un peu prétentieux de comparer notre jeu à Halo, le but du projet n'est pas réellement le jeu en lui-même : en effet, l'utilisation d'OpenGL pour avoir des mécaniques et fonctionnalités de jeu vidéo est l'essence de ce dernier. C'est pourquoi on s'est plutôt orienté vers un petit jeu plus personnel au fur et à mesure, qui se détache un peu (beaucoup) de Halo.



[<https://www.youtube.com/watch?v=0KzAP5NUYqo>]

Nos objectifs étaient les suivants : développer “le jeu en lui-même, les fonctionnalités propres au jeu, c’est-à-dire les mécaniques de gameplay, tous les algorithmes influant sur le rendu et l’UI”. On a ensuite créé une *roadmap* pour nous suivre dans notre développement :

[1] Affichage d’un monde 3D

[2] Exploration du monde avec une vue 1ère personne volante

[3] Affichage du joueur lié à la caméra

[3.1] Joueur visible

[3.2] Arme du joueur visible

[4] Gestion des collisions entre le joueur et le monde.

[5] Mécaniques de déplacement

[5.1] Marche

[5.2] Course

[5.3] Saut

[6] Physique

[6.1] Gravité (joueur)

[7] Mécanique de tir

[7.1] Animation de tir

[7.2] Dégâts sur cibles

[8] Implémentation des ennemis + dégâts sur ennemis

[9] Implémentation des dégâts des ennemis

[10] Implémentation de l’UI

[10.1] Niveau des munitions de l’arme

[10.2] Minimap

[10.3] Nombre d'ennemi tué

[10.4] Pièces

[11] Game design

[11.1] Mise en place de petites énigmes 5

[11.2] Mini combat de boss

[12] Animations du personnage et des ennemis

[13] Expansion du monde

[14] Shaders

[15] Ramassage d'objet au sol

Cette première version contient des éléments que nous n'avons pas fait, pour des raisons que nous évoquerons lors de ce rapport. L'ordre de réalisation des tâches ne reflète pas l'ordre exact du déroulé du projet, mais le but de cette dernière était surtout de nous donner des objectifs, un ordre qui permettrait de nous y référer si nous ne savions pas sur quoi avancer, etc...

Nous avons envie d'essayer de faire un maximum de choses, et surtout de regarder toutes les fonctionnalités d'OpenGL.

II Libraries

On va rapidement expliquer les librairies utilisées pour ce projet. Ces dernières ont été rajoutées au fur et à mesure (mais au final, toutes ont été là très vite) en fonction des besoins grandissants.



On utilise de façon évidente OpenGL, et comme librairie pour la gestion de la fenêtre GLFW, qui a finit par remplacer GLUT lors de la deuxième semaine du projet, car cette dernière permet de contrôler la position du curseur, et donc de le fixer au milieu de l'écran et de l'effacer, comme dans un fps, ce que GLUT ne permettait pas. On a aussi ajouté GLM, librairie mathématique basée sur OpenGL, pour éviter de ré-implémenter des fonctions déjà existantes.

L'utilisation d'OpenGL se fait avec glad, qui permet l'utilisation des fonctions d'OpenGL, et surtout nous avons offert une version récente d'OpenGL, qui inclut les shaders et encapsuleur VAO... dont nous parleront plus tard (Partie Monde et affichage).

Enfin, nous avons utilisé stb pour charger des images, utile seulement pour le chargement de textures.

III Monde et affichage

On va maintenant expliquer le fonctionnement du monde et tout ce qui se rapporte à son affichage.

Objet, texture et affichage :

Les 2 premières choses que nous avons voulu développer lors de notre projet sont les déplacements du joueur et l’affichage du monde. Ce dernier point a évolué au fur et à mesure du projet, pour satisfaire nos attentes sur son rendu.

C’est dans cet objectif qu’on a commencé par implémenter une classe *Objet3D* et une méthode *LoadOBJ*, qui permet de stocker les points, les faces, les vecteurs normaux et de texture à partir d’un fichier *.obj* .

On a choisi de faire nous même cette fonction après avoir envisagé une librairie, Assimp, mais qui n’était pas compatible avec GLUT.

```
std::ifstream file(filename);
std::string line;

if (!file.is_open()) {
    std::cerr << "Failed to open file: " << filename << std::endl;
    return;
}

while (std::getline(file, line)) {
    std::istringstream iss(line);
    std::string token;
    iss >> token;

    if (token == "v") { // on ajoute un format du type v x y z
```

Le seul problème de cette méthode est le fait qu’elle ne supporte que les objets à face triangulaire. Si une face avec plus de sommet est détectée, ses autres points ne sont juste pas stockés et une partie de la face sera affichée.

On a ensuite ajouté une méthode *LoadTexture* pour paramétrer OpenGL pour pouvoir utiliser notre texture en temps voulu.

```

// Load the image using STB Image
int width, height, channels;
unsigned char* image = stbi_load(path, &width, &height, &channels, 0);

if (!image) {
    cout << "Failed to load texture: " << path << endl;
    return;
}

GLuint textureID;
glGenTextures(1, &textureID);
glBindTexture(GL_TEXTURE_2D, textureID);

// Set texture parameters for basic rendering
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);

// Load the image data into the texture
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, width, height, 0, GL_RGBA, GL_UNSIGNED_BYTE, image);

// Free the image data
stbi_image_free(image);
Texture tex;
tex.setID(textureID);
texture = tex;

```

Pour cela, on a utilisé la librairie stb comme expliqué plus haut. Même si les premiers essais n'ont pas posé de soucis, on s'est aperçu plus tard, en cherchant un écran de mort pour le joueur, que toutes les images n'étaient pas chargées pareil. La fonction `glTexImage2D` ne supportait pas certaines images, même si ces dernières étaient bien chargées par notre librairie. On a toujours pas trouvé à quoi cela est dû, mais on a réussi à trouver un écran de mort supporté.

La dernière étape était l'affichage de cet objet. Pour cela, on utilise les fonctions classiques :

```

glBindTexture(GL_TEXTURE_2D, texture.getID());
glBegin(GL_TRIANGLES);
for (const Face& face : faces) { // affichage de la scène

    // Gris

    //glColor3f(vertices[face.v1 - 1].r, vertices[face.v1 - 1].v, vertices[face.v1 - 1].b);
    glTexCoord2f(textureCoord[face.vt1 - 1].v1, textureCoord[face.vt1 - 1].v2);
    glVertex3f(vertices[face.v1 - 1].x, vertices[face.v1 - 1].y, vertices[face.v1 - 1].z);
    //glColor3f(vertices[face.v2 - 1].r, vertices[face.v2 - 1].v, vertices[face.v2 - 1].b);
    glTexCoord2f(textureCoord[face.vt2 - 1].v1, textureCoord[face.vt2 - 1].v2);
    glVertex3f(vertices[face.v2 - 1].x, vertices[face.v2 - 1].y, vertices[face.v2 - 1].z);
    //glColor3f(vertices[face.v3 - 1].r, vertices[face.v3 - 1].v, vertices[face.v3 - 1].b);
    glTexCoord2f(textureCoord[face.vt3 - 1].v1, textureCoord[face.vt3 - 1].v2);
    glVertex3f(vertices[face.v3 - 1].x, vertices[face.v3 - 1].y, vertices[face.v3 - 1].z);
}

glEnd();

```

Pas de soucis particulier, et nous voici avec un moyen d'afficher n'importe quel objet dans notre monde.

IV UI

On peut maintenant créer l'UI du joueur. Ce dernier comprend : l'affichage de l'arme principale et du couteau, un crosshair, l'affichage de sa vie, de ses munitions et de l'écran de mort.

Pour cela, on a créé une classe UI, qui contient une liste d'*Objet3D*, avec un état concernant leur activité, ainsi qu'une méthode pour les afficher.

Le plus compliqué a été l'affichage de ces derniers : en effet, on devait avoir tous ces modèles 3D de façon fixe par rapport à la caméra. On a eu quelques soucis, mais on a trouvé (et par erreur!) la solution :

```
glPushMatrix();// On sauvegarde la matrice actuelle,
vu que l'on veut effectuer des modifs de cette
matrice seulement pour l'UI
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity(); // On reset la matrice de vue
à l'identité, comme ça on affiche par rapport à la
caméra et les objets restent ainsi fixes par rapport
à la caméra
    Ui.affichage(keys, mouseClick,
healthShader,alphaShader,camera_position,camera_cent
er_vector,playerHealth,actFrame);
    glPopMatrix();// On reprend la matrice qui était
présente avant l'affichage de l'UI
```

L'idée est de placer comme matrice modèle actuelle celle de la caméra. Ainsi, les objets sont dessinés par rapport à la caméra, qui est notre joueur, et donc de manière fixe par rapport à lui. Il suffit de bien les placer avec des translations, et le tour est joué.



Ici, voici un état de notre UI avec la vie du joueur

A COMPLETER

Là, avec la barre de munition et l'arme



Et enfin, l'écran de mort.

On a également ajouté des animations : l'animation de coup de couteau, de course pour le couteau et l'arme, et un peu de mouvement aux cœurs. Ce sont des animations faites purement à la main, c'est à dire implémentées avec des rotations/translations OpenGL et non pas des animations importées. Ce n'est pas quelque chose que nous avons eu le temps de faire.

V Shaders

Malheureusement, on s'est rendu compte que cette seule méthode d'affichage était un peu limitée. En effet, on n'a pas beaucoup d'options de customisation pour l'affichage. On a alors su qu'il fallait améliorer notre version d'OpenGL, et surtout changer notre méthode d'affichage :

En OpenGL moderne, il existe un système de VAO, EBO et VBO. VAO est une référence à un objet avec une liste de points, la liste des indices de ces points, et idem pour les coordonnées de texture. VAO représente l'objet entier, EBO les indices et VBO les coordonnées des points de l'objet, avec leurs coordonnées de texture et leur vecteurs normaux.

```
glGenVertexArrays(1, &VAO);
glGenBuffers(1, &VBO);
glBindVertexArray(VAO);
glGenBuffers(1, &EBO);
// on set notre liste d'indices en 0, 1, 2, ... pointsTexture.size() car ils sont déjà dans l'ordre d'affichage
for (int i = 0; i < pointsTexture.size(); i++) {
    indices.push_back(i);
}
glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO);
glBufferData(GL_ELEMENT_ARRAY_BUFFER, indices.size() * sizeof(GLuint), indices.data(), GL_STATIC_DRAW);
glBindBuffer(GL_ARRAY_BUFFER, VBO);
glBufferData(GL_ARRAY_BUFFER, pointsTexture.size() * sizeof(PointText), pointsTexture.data(), GL_STATIC_DRAW);
glEnableVertexAttribArray(0);
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, sizeof(PointText), (void*)offsetof(PointText, position));
glEnableVertexAttribArray(1);
glVertexAttribPointer(1, 2, GL_FLOAT, GL_FALSE, sizeof(PointText), (void*)offsetof(PointText, u));
glEnableVertexAttribArray(2);
glVertexAttribPointer(2, 2, GL_FLOAT, GL_FALSE, sizeof(PointText), (void*)offsetof(PointText, normal));
glBindBuffer(GL_ARRAY_BUFFER, 0);
glBindVertexArray(0);
```

Comme le montre le code ci-dessus, après avoir activé ces 3 éléments, on leur donne la liste d'indice et de point, en spécifiant comment, à partir de notre structure *pointText* récupérer la position, la normal et les coordonnées de texture, et après, on a plus qu'à garder notre VAO.

On est maintenant capable d'afficher un objet avec cette méthode :

```
glm::mat4 model = glm::mat4(1.0f);
glm::mat4 modelF = glm::mat4(1.0f);
modelF = glm::translate(model, offset);
glm::mat4 view = glm::lookAt(cameraPosition, cameraTarget, cameraUp);

float fov = glm::radians(90.0f); // Field of view in radians
float aspectRatio = 800.0f / 600.0f; // Width divided by height
float nearClip = 0.1f;
float farClip = 10000.0f;

// Create the projection matrix
glm::mat4 projection = glm::perspective(fov, aspectRatio, nearClip, farClip);

// Use the shader program and set the model matrix as a uniform.
glUseProgram(shader.getShader());
glUniformMatrix4fv(glGetUniformLocation(shader.getShader(), "model"), 1, GL_FALSE, glm::value_ptr(modelF));
glUniformMatrix4fv(glGetUniformLocation(shader.getShader(), "view"), 1, GL_FALSE, glm::value_ptr(view));
glUniformMatrix4fv(glGetUniformLocation(shader.getShader(), "projection"), 1, GL_FALSE, glm::value_ptr(projection));
glUniform1i(glGetUniformLocation(shader.getShader(), "texture1"), 0);

glBindTexture(GL_TEXTURE_2D, texture.getID());
//Display
glBindVertexArray(VAO);

glDrawElements(GL_TRIANGLES, pointsTexture.size(), GL_UNSIGNED_INT, 0);
//End of display
glBindVertexArray(0);
glUseProgram(0);
glBindTexture(GL_TEXTURE_2D, 0);
```

On est capable, avec un shader, d'afficher notre objet avec texture. On peut également modifier aisément les matrices de vue, modèle et de projection à notre guise.

L'avantage avec ceci, est que nous avons maintenant un affichage bien plus customisable.

Par exemple, on peut créer une skybox!



L'idée est assez simple : on crée un cube que l'on importe en tant qu'*Objet3D*, et on lui donne un shader particulier :

```
const char* vertexShaderSourceSkyBox = R"(
    #version 330 core
    layout(location = 0) in vec3 aPos;
    layout(location = 1) in vec2 aTexCoord;
    uniform mat4 model;
    uniform mat4 view;
    uniform mat4 projection;
    out vec2 TexCoord;
    out vec3 pos;
    void main() {
        gl_Position = projection *view *model *
vec4(aPos, 1.0);
        TexCoord = aTexCoord;
        pos = aPos;
    }
)";

const char* fragmentShaderSourceSkyBox = R"(
    #version 330 core

    out vec4 FragColor;
    in vec3 pos;
    const vec4 skytop =vec4(0.0f, 0.0f, 0.2f, 1.0f);
    const vec4 skyhorizon = vec4(1.0f, 0.5f, 0.0f,
1.0f);

    void main() {
        vec3 pointOnSphere = normalize(pos);
        float a = min(pointOnSphere.y+0.8,1.0);

        FragColor =mix(skyhorizon, skytop, a);

    }
)";
```

Le vertex shader, l'affichage des points, est celui de base. Néanmoins, dans le fragment Shader, on peut modifier la couleur du pixel en fonction de sa hauteur et lui donner un dégradé entre une couleur d'horizon couleur crépuscule et une couleur ciel nuit. L'avantage du shader, au lieu d'utiliser une texture, est que celle-ci est facilement customisable. Au début, la skybox était couleur ciel jour et on a pu la changer très facilement avec cette implémentation, puisqu'il suffit de changer les deux couleurs dans le shader.

Cela peut aussi permettre un affichage utile au joueur, notamment sur des indications visuels, mais nous en reparlerons lorsque nous aborderons l'IA.

VI Joueur et fonctionnalités

Maintenant, on va parler du joueur et de ce qu'il peut faire à partir du début.

Bien évidemment, il peut se déplacer avec ZQSD, sur la carte. Pour permettre au joueur de rester constamment sur le plateau, on a utilisé un NavMesh et du rayCasting. Pour cela, on a créé une classe *rayon*, qui permet, à partir d'un objet 3D, d'une direction et d'un point de départ, de détecter une éventuelle collision.

```
bool RayIntersectsTriangle(glm::vec3 rayOrigin,
    glm::vec3 rayVector,
    glm::vec3 vertex0,
    glm::vec3 vertex1,
    glm::vec3 vertex2,
    glm::vec3& outIntersectionPoint)
{

    const float EPSILON = 0.0000001;
    glm::vec3 edge1, edge2, h, s, q;
    float a, f, u, v;
    edge1 = vertex1 - vertex0;
    edge2 = vertex2 - vertex0;
    h = glm::cross(rayVector, edge2);
    a = glm::dot(edge1, h);
    if (a > -EPSILON && a < EPSILON)
        return false; // si parallèle
    f = 1.0 / a;
    s = rayOrigin - vertex0;
    u = f * dot(s, h);
    if (u < 0.0 || u > 1.0)
        return false;
    q = cross(s, edge1);
    v = f * dot(rayVector, q);
    if (v < 0.0 || u + v > 1.0)
        return false;

    float t = f * dot(edge2, q);
    if (t > EPSILON || t < -EPSILON) // intersection
    {
```

```

        outIntersectionPoint = rayOrigin + rayVector * t;

    return true;
}
else {

    return false; }// pas d'inter

}

```

La fonction précédente renvoie s'il y a eu ou non intersection entre un triangle et un rayon, et change le point d'intersection passé en référence. C'est l'algorithme de Möller-Trumbore.

On l'utilise ensuite dans plusieurs fonctions déclinées pour vérifier l'intersection sur un objet entier composé de multiples triangles.

Ensuite, à chaque frame, on va asservir la position du joueur en Y (hauteur) en envoyant un rayon depuis le joueur vers le bas. On prend le point d'intersection et on met le joueur à cette position plus sa taille.

La méthode marche plutôt bien, mais est vite dépassée si le navMesh contient beaucoup de triangles. Une méthode à laquelle nous avons pensé, mais pas implémenté car le jeu est assez petit, est de segmenter notre monde en carré : on connaît la position des coins de chaque carrés, et donc, en connaissant la position du joueur, on sait dans quelle carré il se trouve et donc sur quels triangles appliqués notre algorithme. L'idée nous est venue du jeu Pokémon Platine, qui utilise ce système de map pour l'optimisation, mais pour l'affichage cette fois-ci.

Le joueur est donc maintenant capable de se déplacer!

Il possède également une fonction de saut : Lors de son activation, on déclenche un timer qui permet d'asservir sa position en y en fonction. On désactive le raycast pendant ce temps, et on le réactive pendant la phase descendante.

```

void Camera::updateJump(float intersectiony)
{

```

```

if (jump.getIsJumping())
{

    jump.actualizeJump();

    if (intersectiony > (jump.getYCoord()-4.0)
    && jump.isGoingDown()) { // on est en dessous du
navMesh

        jump.resetJump();

    }
    else { // on peut continuer à descendre
        // on remet bien la target de la
caméra au bon endroit
        float delta = camera_position.y;
        camera_position.y =
jump.getYCoord();
        delta = camera_position.y - delta;

        camera_center_vector.y += delta;

    }

}

}

```

Ici, le principe est le suivant : lorsque le joueur est en mode saut et descend , il est censé être au-dessus du NavMesh. Ainsi, on attend le moment où il passe d'un état où l'on détecte une face en dessous à un état où l'on ne détecte rien : cela veut dire qu'on vient de passer en dessous du sol. Alors, on désactive le saut et on le met à sa dernière position au-dessus du sol.

VII Interactions avec le monde

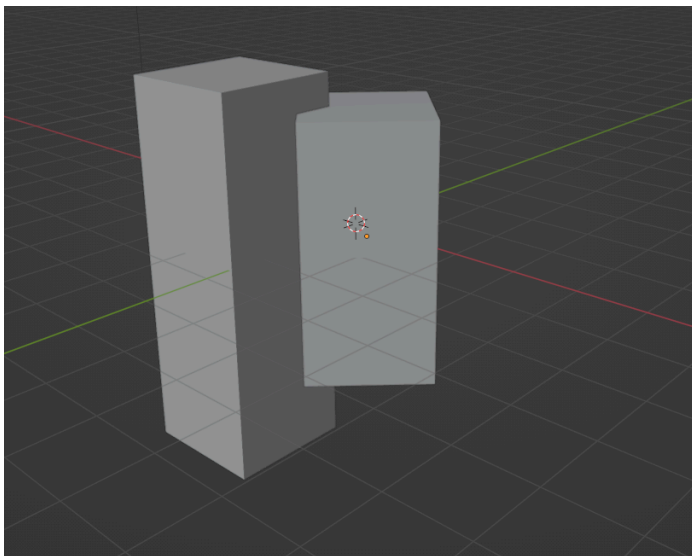
Il nous reste également à voir comment le joueur interagit avec le monde. On a commencé par implémenter un système de collider, que nous avons créé nous même.

Le gestion des collisions est utilisé à partir d'une méthode sur un Collider, qui prend en entrée un autre Collider.

On commence avec une fonction qui vérifie si un point est dans un modèle 3D ou non. A vrai dire, pour notre jeu, on n'utilise que des modèles rectangulaires de Collider par souci de simplicité. Pour vérifier si un point est à l'intérieur, on projette un rayon (grâce à la classe rayon implémenté auparavant) selon les 3 directions (X,Y et Z) et on regarde si les 3 rayons touchent nos faces de rectangle. Cela donne un bon résultat pour un rectangle, et pour toute structure convexe à vrai dire.

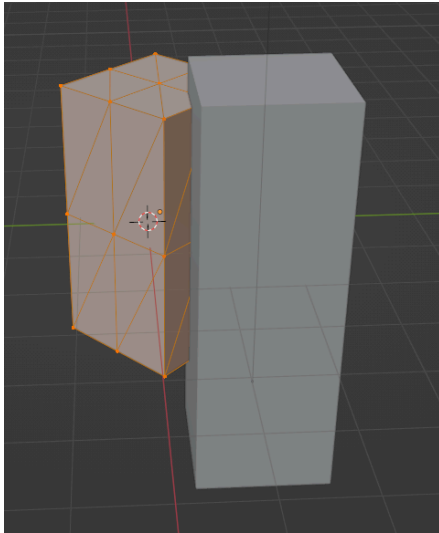
L'idée consiste donc à vérifier si un point du modèle d'entrée se trouve à l'intérieur du Collider sur lequel on applique la méthode. Si c'est la cas, alors il y a collision.

Néanmoins, cela ne marche pas tout le temps :



Si on applique la méthode sur le plus grand rectangle, on voit qu'aucun point de l'autre rectangle n'est à l'intérieur de l'autre, et pourtant il y a collision.

Il est donc nécessaire d'avoir un rectangle en entrée un minimum raffiné si celui ci :



Ici, une seule subdivision supplémentaire suffit à changer l'état de non-collision à un état de collision. Et souvent, un raffinement suffit (nos rectangles ne vont pas dans tous les sens possibles non plus).

C'est une solution suffisante pour ce qu'on compte en faire.

Ainsi, on vérifie avant d'avancer qu'il n'y a pas de collision dans la position suivante, sinon on empêche le joueur d'avancer.

On a également la méthode de tir :

Assez simple une fois que nous avons la classe rayon : on projette un rayon du joueur vers la direction selon laquelle il regarde, et on regarde s'il y a une intersection avec un ennemi dans une liste d'ennemis, et on lui enlève des points de vie. On vérifie tout de même que le joueur possède assez de munitions. Lorsqu'il tire depuis le couteau, on vérifie qu'il est assez proche de l'ennemi avec un calcul de distance.

VIII IA ennemi

On va finir par expliquer l'IA que nous avons implémenté pour l'ennemi.

Voici le principe de fonctionnement : le boss possède 3 zones : une zone courte portée, moyenne portée et longue portée. Celles-ci sont définies par un rayon autour de la position du boss. Quand le joueur se trouve sur le cercle extérieur, il déclenche une attaque longue distance, qui est une chute de météorite. S'il se trouve sur le cercle du milieu, le boss poursuit le joueur et le cercle extérieur se rapproche petit à petit du cercle intérieur pour déclencher soit l'attaque longue portée, soit l'attaque courte portée. Cette dernière se déclenche une fois le joueur entré dans le cercle intérieur. A partir de ce moment, un timer est déclenché et le cercle intérieur voit son rayon doublé. A la fin du timer, si le joueur se trouve dans le cercle intérieur, il perd de la vie. Cette durée est représentée par la disparition du boss, qui une fois complètement réapparu lance son attaque.

A présent, on va expliquer le fonctionnement de l'IA avec le code. On dispose de la classe *MovementManager* suivante :

```
#pragma once

#include "Ennemi.h"
#include "Camera.h"
#include "glm/glm/glm.hpp"
#include "PatternManager.h"

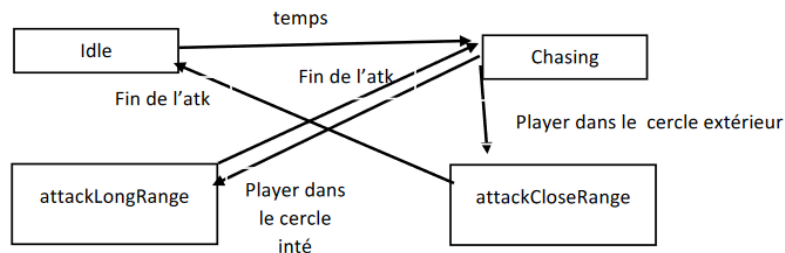
class MovementManager
{
protected:
    Ennemi* obj;
    bool chasing;
    bool idle;
    float tIdle;
    bool attackCloseRange;
    float closeRange;
    bool attackLongRange;
    float longRange;
    float longCircleRadius;
    bool meteorChase;
    float closeCircleRadius;
public:
    MovementManager(Ennemi* o);
    void updatePos(Camera &c, PatternManager &p);
    float getLongRadius();
    float getCloseRange();
    float getCloseRadius();
};
```

Celle-ci permet de définir 4 états : idle, chasing, attackLongRange et attackCloseRange. Si idle est un état stationnaire qui permet de

laisser un petit répit au joueur, les 3 autres sont assez explicites. On possède également des timers pour chaque état qui permette, par exemple pour l'idle, d'en sortir au bout d'un certain temps, ou de rapprocher le cercle extérieur pendant le mode chasing.

On peut alors définir le diagramme suivant :

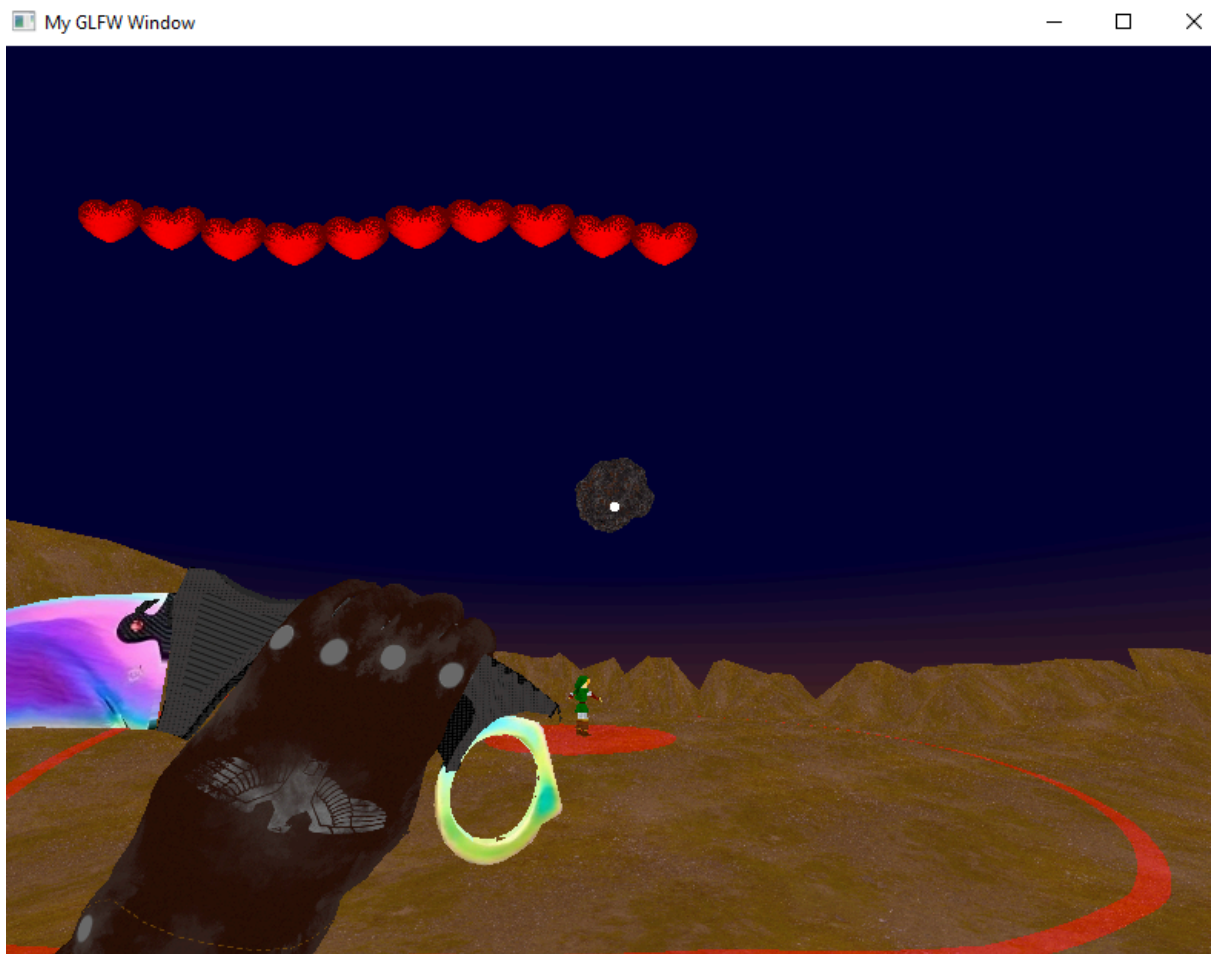
On peut ainsi définir un diagramme :



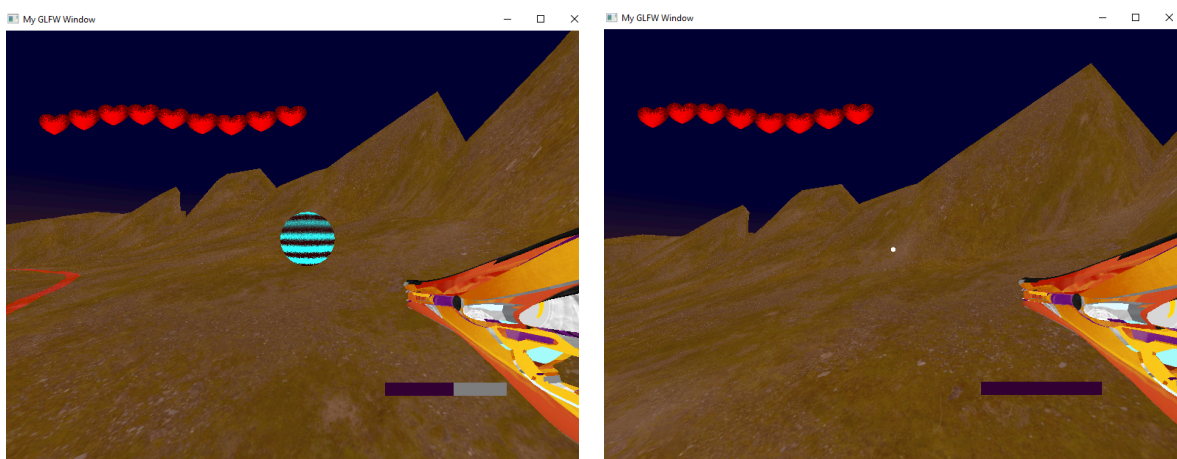
La chute de météorite, dont nous avons parlé juste plus haut, qui correspond à l'attaque longue distance, se décompose ainsi : une météorite tombe du ciel et suit le joueur jusqu'à ce qu'elle atteigne une certaine hauteur, à partir de laquelle elle suit sa trajectoire. On utilise les collisions pour savoir si elle touche le joueur.

C'est d'ailleurs là que l'on voit l'utilité de nos shaders : on va être capable d'afficher les cercles de moyenne et longue portée, ainsi que changer le rayon du cercle moyenne portée lors de l'attaque chargée pour prévenir le joueur. On va également mettre l'ombre du météore sur le sol pour avertir le joueur.

On peut voir la météorite, ainsi que les cercles sur l'image suivante.



Enfin, on a des orbes qui permettent au joueur de recharger sa barre de munition



On voit l'orbe dans la photo ci-dessus, et elle permet de recharger sa barre de munitions.

Enfin, on a ajouté un système pour permettre à l'ennemi de toujours nous regarder en face, à l'aide d'un angle de rotation mis à jour en fonction de sa position et de celle du joueur.

IX Conclusion

Pour conclure, on est assez fier de notre avancement compte-tenu de la road map que l'on s'était fixé. On a aussi remarqué que sur les dernières semaines, l'implémentation des nouvelles fonctionnalités s'est effectuée assez rapidement avec peu de debug, contrairement au tout début, où chaque ajout nécessitait un long temps de débogage. Cela témoigne de l'aisance que nous avons acquise sur OpenGL, mais aussi du projet qui devenait plus stable et plus structuré avec un code plus complet et plus solide.

C'est aussi un projet qui nous tenait à cœur au début et qui nous a beaucoup plu, étant tous deux intéressés par le développement de jeux vidéos, et nous sommes content d'avoir pu investir du temps dans ce projet.